Running a Graph

Overview

- **We have 3 options:**

  1️⃣ Run Graphs in .ipynb

  2️⃣ Expose Graphs on a Local Server

  3️⃣ Expose Graphs on a Deployment Server

- To use `LangSmith Studio UI` or `LangSmithAgent Chat UI`, the graph must be **exposed**
- But independent of if our graph is **exposed** or just running from a .ipynb, we will get traces sent to LangSmith if we set:

```
export LANGSMITH_TRACING=true
export LANGSMITH_API_KEY="your_key"
export LANGSMITH_PROJECT="my-project-name"
```

LangSmith Studio UI

- **LangSmith Studio UI provides:**
  - A chat-style UI for running the graph and providing inputs
  - A live, interactive view of the graph diagram as it executes
  - Step-by-step state inspection and replay controls
- **To setup, we must expose our graph. See above pages 2️⃣ & 3️⃣**

LangSmith Agent Chat UI

- LangSmith Agent Chat UI is meant to be a user-facing front end which we can send the link to users

`Invoke` Vs `Stream`

We use `invoke` or `stream` whenever we run a graph programattically (either in .ipynb or hosted)

Synchronous vs Asynchronous

- Sync:
  - You do one thing at a time.
  - You cannot start the next task until the current one finishes.
  - **Here: You watch tokens come in one after the other, and your program waits until each chunk arrives before doing anything else.**
- Async:
  - You can start something, let it run in the background, and do other things meanwhile.
  - **Here: Your program can receive tokens while also doing other work.**
    - i.e. if there are other instances doing the same they could run whilst we are waiting
    - More often used in production than in local notebooks

- `.invoke` and `.ainvoke` are for full, one-shot execution.
  - both run the graph once and return the full state at the end once everything is done
  - No streaming, no partial results.
- `.stream` and `.astream` are for incremental streaming.
  - Run the same graph, but **yield stuff while it's running**.

    - I.e. the new parts of state after a node, or the new parts of state token-by-token
- What you get depends on `stream_mode`: `values`, `updates`, `custom`, `messages`, `debug`.

Mode	Description
`values`	After each step (usually after nodes finish), you get a **full snapshot** of the state.
`updates`	After each step (usually after nodes finish) you get only what changed in the state.
`messages`	Used to stream tokens 2-tuples. Specifically, we get (LLM token, metadata) tuples from any graph node where an LLM is invoked.
`debug`	Streams as much information as possible throughout the execution of the graph.
`custom`	You decide.

▼ `Updates` **Stream Demo**

```
# ...
# compile graph

# Pass in te input to the graph and then have it stream response
for chunk in graph.stream(
    {"topic": "ice cream"},
    # Set stream_mode="updates" to stream only the updates to th
e graph state after each node
    # Other stream modes are also available. See supported stream
modes for details
    stream_mode="updates",
):
    print(chunk)

# THIS IS THE OUTPUT OF THE refineTopic node
{'refineTopic': {'topic': 'ice cream and cats'}}
```

```
# THIS IS THE OUTPUT OF THE generateJoke node
{'generateJoke': {'joke': 'This is a joke about ice cream and cats'}}
```

- To include outputs from **subgraphs** in the streamed outputs, you can set `subgraphs=True` in the `.stream()` method of the parent graph
  - The outputs will be streamed as tuples `(namespace, data)`, where `namespace` is a tuple with the path to the node where a subgraph is invoked, e.g. `("parent_node:<task_id>", "child_node:<task_id>"):`

```
((), {'node_1': {'foo': 'hi! foo'}})
(('node_2:dfddc4ba-c3c5-6887-5012-a243b5b377c2',), {'subgraph_node_1': {'bar': 'bar'}})
(('node_2:dfddc4ba-c3c5-6887-5012-a243b5b377c2',), {'subgraph_node_2': {'foo': 'hi! foobar'}})
((), {'node_2': {'foo': 'hi! foobar'}})
```

▼ `Messages` Stream Demo

- Use the `messages` streaming mode to stream Large Language Model (LLM) outputs **token by token** from any part of your graph, including nodes, tools, subgraphs, or tasks.

The streamed output from `messages` mode is a tuple (message_chunk, meteadata) where:

- `message_chunk`: the token or message segment from the LLM.
- `metadata`: a dictionary containing details about the graph node and LLM invocation.

🚨 If your LLM is not available as a LangChain integration, you can stream its outputs using `custom` mode instead. See **use with any LLM** for details.

```
# ...
# compile graph

for message_chunk, metadata in graph.stream(
    {"topic": "ice cream"},
    stream_mode="messages",
):
    if message_chunk.content:
        print(message_chunk.content, end="|", flush=True)
```

```
for msg, metadata in graph.stream(
    inputs,
    stream_mode="messages",
):
    # Filter the streamed tokens by the langgraph_node field in the
metadata
    # to only include the tokens from the specified node
    if msg.content and metadata["langgraph_node"] == "some_nod
e_name":
        ...
```